Random Forest

Learning Journal & Experiment Report

Understanding Random Forest from the Inside

Master 1 Artificial Intelligence · Machine Learning · Practical Experiments

Author: **Aissa Aftis**

Learning path: Decision Trees → Random Feature Selection → Bootstrap Sampling → Error Diversity → Majority Voting → Generalization

1. Introduction

The objective of this learning session was not simply to use a Random Forest implementation from Scikit-learn. The objective was to understand *why* the algorithm works, how its internal mechanisms affect its predictions, and how to investigate its behavior through experiments.

During this work, I explored the relationship between individual tree accuracy, tree diversity, correlated errors, majority voting, and the number of trees.

The most important discovery was:

> **A Random Forest can outperform its individual trees because it combines their predictions, especially when their errors are not strongly correlated.**

However, adding more trees or increasing diversity does not guarantee better test accuracy. The quality of the individual trees and the structure of their errors both matter.

2. Decision Trees: The Foundation

A Decision Tree recursively divides the feature space into smaller regions by selecting splits that improve class separation. Two important impurity measures are Gini impurity and entropy.

2.1 Gini impurity

For a classification node containing K classes:

$$Gini = 1 - \Sigma_{k=1..K} p_k^2$$

where p_k is the proportion of samples belonging to class k. A node containing only one class has Gini impurity equal to zero.

2.2 Entropy

$$H(S) = - \Sigma_{k=1..K} p_k \log_2(p_k)$$

Entropy measures the uncertainty in a node. A node containing samples from only one class has entropy zero.

2.3 Information gain

When a candidate split divides a parent node into left and right children, information gain is:

$$IG = H(S) - (|S_L|/|S| \cdot H(S_L) + |S_R|/|S| \cdot H(S_R))$$

The tree searches for a split that produces a useful reduction in impurity.

> **My discovery:** The quality of a Decision Tree depends heavily on the splits it chooses. Different training samples or different candidate features can produce different tree structures.

3. How Random Forest Works

A Random Forest combines multiple Decision Trees. Each tree is generally trained using two sources of randomness.

- **1. Bootstrap Sampling** – Training examples are sampled with replacement. Some observations may appear multiple times, while others may not appear in a particular tree's training sample.
- **2. Random Feature Selection** – At each node, the algorithm considers a random subset of features when searching for the best split.
- **3. Ensemble Prediction** – For classification, the forest usually selects the class receiving the most votes from the trees.

These mechanisms encourage different trees to learn different patterns. Their purpose is not to make every tree perfect, but to construct an ensemble whose combined prediction is reliable.

4. My First Important Question: Which Errors Are Better?

Question: Which situation helps Random Forest more?

- **Situation A:** all trees make errors on the same samples.
- **Situation B:** trees make errors on different samples.

My answer: B.

Situation A — correlated errors

Imagine five trees making the same mistakes:

Tree	Sample 1	Sample 2	Sample 3
Tree 1	**Wrong**	**Wrong**	Correct
Tree 2	**Wrong**	**Wrong**	Correct
Tree 3	**Wrong**	**Wrong**	Correct
Tree 4	**Wrong**	**Wrong**	Correct
Tree 5	**Wrong**	**Wrong**	Correct

The majority vote is wrong on Samples 1 and 2. All five trees fail on the same examples, so voting cannot correct these mistakes.

Situation B — different errors

Tree	Sample 1	Sample 2	Sample 3
Tree 1	**Wrong**	Correct	Correct
Tree 2	Correct	**Wrong**	Correct
Tree 3	Correct	Correct	**Wrong**
Tree 4	Correct	Correct	Correct
Tree 5	Correct	Correct	Correct

The majority vote is correct on all three samples.

Conclusion: Random Forest benefits when trees make different errors, because correct trees can outvote incorrect ones. However, diversity alone is insufficient. The trees must still provide useful predictions.

5. My Second Question: How Can 20 Wrong Trees Still Produce a Correct Prediction?

Suppose a Random Forest has 50 trees:

- 30 trees predict **Normal**.
- 20 trees predict **DDoS**.

If the true class is Normal, the final prediction is correct because Normal receives more votes. The vote proportions are:

$$P(\text{Normal vote}) = 30/50 = 0.60$$

$$P(\text{DDoS vote}) = 20/50 = 0.40$$

The forest predicts Normal.

My discovery: Every tree does not need to be correct. The ensemble needs enough correct votes to overcome the incorrect votes. For multiclass classification, the same principle applies: the class with the largest vote count wins, regardless of how many individual trees predicted other classes.

6. Feature Correlation vs. Error Correlation

One of my hypotheses was that trees might choose the same features because those features are highly correlated. This is a useful hypothesis, but two different concepts must be distinguished.

Concept	Definition	Example
Feature correlation	Input features contain related information.	height_cm and height_m
Prediction agreement	Trees predict the same class on a sample.	Two trees both predict Normal
Error correlation	Trees tend to make mistakes on the same samples.	Both trees misclassify the same attack

Feature correlation can encourage trees to make similar splits, but it does not guarantee that they will be identical. Bootstrap samples, random feature selection, and different split thresholds can still produce different trees.

Another important distinction is that prediction agreement does not necessarily mean error correlation. Two trees can agree because both are correct, or because both are wrong. For ensemble performance, the pattern of errors is especially important.

7. Understanding max_features

The `max_features` parameter controls how many features are considered at each node.

Setting	Meaning	Typical effect
1	One randomly selected feature	Often more diversity, but potentially weaker splits
2	Two candidate features	A compromise depending on the dataset
"sqrt"	Approximately the square root of the feature count	Common classification setting
None	All available features	Potentially stronger splits, but trees may be more similar

Reducing max_features generally encourages diversity, but this is not guaranteed to improve accuracy. Why?

- 1. Fewer candidate features can force trees to explore different splits.
- 2. Different splits can make their predictions less correlated.
- 3. However, a useful feature might not be available at a particular node.
- 4. The resulting tree can become weaker.
- 5. The forest must balance individual-tree strength against ensemble diversity.

This is a practical example of the bias–variance trade-off.

My experiment

I tested different max_features settings, including 1, 2, "sqrt" and None. I reported 1.0000 for training and test accuracy across these settings in my experiment. The important conclusion is not that max_features has no effect. Instead:

> **Equal accuracy does not imply identical trees or identical predictions on every possible dataset.** All the settings may have classified the current test set correctly despite producing different internal trees.

8. My Random Forest Experiments

8.1 Individual tree accuracy

I trained a forest containing 50 trees and evaluated each tree separately on a test set of 30 samples. My observations were:

Individual tree accuracy	Number of trees	Correct samples
100%	27	30/30
96.67%	18	29/30
93.33%	4	28/30
90%	1	27/30

These counts account for all 50 trees. The mean individual-tree accuracy was:

$$\bar{A} = [27(1) + 18(29/30) + 4(28/30) + 1(27/30)] / 50 \approx 0.98067 = \mathbf{98.07\%}$$

My Random Forest achieved 1.0000, or 100%, test accuracy. This means the forest correctly classified all 30 test samples, even though some individual trees made mistakes.

8.2 What I learned from this experiment

- Some trees performed better than others.
- Not every tree needed to achieve 100% accuracy.
- The ensemble could combine the trees' predictions successfully.
- The forest's accuracy exceeded the mean individual-tree accuracy on this test set.

However, a 30-sample test set is small. A result of 100% is encouraging, but it does not prove that the model will classify all future data correctly.

8.3 Prediction agreement

I also obtained the following result:

```
tree_preds.shape
# (50, 30)
```

This means there were 50 trees and 30 test samples. I reported agreement with Tree 0 of approximately 0.9620, or 96.2%. This is a measure of how often other tree predictions matched Tree 0 over the evaluated tree-sample comparisons. It is **not** a direct measurement of error correlation. The forest can achieve 100% accuracy even when some trees disagree with Tree 0, because Tree 0 itself makes mistakes and the ensemble can correct them.

9. Code: Predict with Every Tree

To inspect the individual trees, I added a helper method to my RandomForestScratch class.

```
def predict_each_tree(self, X):
    return np.array([
        tree.predict(X)
        for tree in self.trees
    ])
```

The returned array has one row per tree and one column per test sample. For my experiment:

```
tree_preds = forest.predict_each_tree(X_test)
print(tree_preds.shape)
# (50, 30)
```

This is useful because it allows us to analyze the behavior of the individual trees rather than examining only the final forest prediction.

10. Code: Measure Individual Tree Accuracy

```
tree_preds = forest.predict_each_tree(X_test)

tree_accuracies = np.mean(
    tree_preds == np.asarray(y_test).reshape(1, -1),
    axis=1
)

print("Mean individual tree accuracy:", tree_accuracies.mean())
print("Best tree accuracy:", tree_accuracies.max())
print("Worst tree accuracy:", tree_accuracies.min())
print("Forest accuracy:", np.mean(forest.predict(X_test) == y_test))
```

What this code teaches

- `axis=1` calculates accuracy separately for each tree.
- `tree_accuracies.mean()` calculates the mean accuracy of the individual trees.
- `tree_accuracies.max()` identifies the highest individual-tree accuracy.
- `tree_accuracies.min()` identifies the lowest individual-tree accuracy.
- The last expression calculates the accuracy of the ensemble itself.

Comparing these values helps determine whether the forest improves on the average individual tree.

11. Code: Analyze Errors Per Sample

My next experiment is to count how many trees make mistakes on each test sample.

```
errors = (
    tree_preds !=
    np.asarray(y_test).reshape(1, -1)
)

errors_per_sample = errors.sum(axis=0)

print("Errors per sample:")
print(errors_per_sample)

print("Samples where all trees are correct:",
      np.sum(errors_per_sample == 0))

print("Samples where all trees are wrong:",
      np.sum(errors_per_sample == len(forest.trees)))

print("Samples with a mixture of correct and wrong trees:",
      np.sum((errors_per_sample > 0) &
             (errors_per_sample < len(forest.trees))))
```

This code counts the number of incorrect trees for each sample. For example, if a sample has 20 incorrect tree predictions, then 30 trees are correct. If those 30 trees agree on the true class, the forest predicts correctly. For multiclass problems, the error count alone does not reveal the full voting distribution. We can also count how many trees voted for each predicted class.

12. Does Random Forest Always Outperform a Decision Tree?

No. Random Forest often improves stability and generalization by combining multiple trees, but it does not always outperform a single Decision Tree. Consider two scenarios:

- **Scenario A:** A Decision Tree already performs extremely well on representative unseen data. A Random Forest may not improve its accuracy measurably.
- **Scenario B:** A Decision Tree is unstable and sensitive to the training data. A Random Forest may reduce this instability and achieve better generalization.

The appropriate model should be selected using validation or cross-validation results, rather than assuming that an ensemble must always be better.

13. Does Increasing the Number of Trees Improve Accuracy?

My next question was whether increasing the number of trees from 50 to 500 guarantees better performance. The answer is **no**. More trees often make predictions more stable, particularly when the individual trees are not perfectly correlated. However, the test accuracy may remain unchanged, and it is not guaranteed to improve.

Number of trees	Possible outcome
50	Test accuracy = 100%
100	Test accuracy remains 100%
500	Test accuracy remains 100%

These are illustrative outcomes, not guaranteed results. If the forest already achieves 100% on a fixed test set, its accuracy on that set cannot increase further. Additional trees may still change its predictions on other samples or improve stability across repeated experiments. More trees also require more memory and generally increase training and prediction costs.

The variance intuition

For an average of T numeric predictions with equal variance σ^2 and pairwise correlation ρ, a simplified formula is:

$$Var(\bar{f}) = \sigma^2 \left(\rho + (1 - \rho) / T \right)$$

This illustrates two important ideas:

- Increasing T reduces the contribution from the uncorrelated part of the variance.
- If the predictions are highly correlated, the benefit of adding more trees is limited.

This formula is an intuition for averaging numeric predictions under simplifying assumptions. It is not an exact formula for classification accuracy or majority voting.

14. Final Challenge: Identical Accuracy or Identical Models?

I was asked to compare three forests:

- 50 trees: test accuracy of 100%.

- 100 trees: test accuracy of 100%.
- 500 trees: test accuracy of 100%.

My answer was that their predictions may differ despite having the same accuracy. That answer is correct when referring to predictions across other samples or to their internal tree predictions.

There is one important qualification: if all three models achieve 100% accuracy on exactly the same single-label classification test set, then all three must predict the correct class for every sample in that set. They cannot disagree on a test sample while all remaining 100% accurate on that same set. Nevertheless, their tree structures, vote distributions, and predictions on other datasets may differ.

15. Recommended Next Experiment

To deepen my understanding, I can run repeated experiments using different max_features settings and random seeds. For each experiment, I should record:

- Mean individual-tree accuracy.
- Best and worst individual-tree accuracy.
- Forest validation accuracy.
- Forest test accuracy.
- Agreement between trees.
- How often trees make errors on the same samples.
- Training time and prediction time.

I should keep a separate final test set and avoid repeatedly tuning the model against it. Validation data or cross-validation is more appropriate for selecting hyperparameters.

16. Final Conclusions

What I discovered

- **1. Random Forest does not require perfect trees.** The ensemble can correct individual mistakes through voting.
- **2. Diversity matters.** Trees that make different errors can complement one another.
- **3. Feature correlation and error correlation are different.** Similar input features can influence tree similarity, but they do not guarantee identical errors.
- **4. max_features creates a trade-off.** Lower values often increase diversity but may weaken individual splits.
- **5. Equal accuracy does not imply equal models.** Different forests can achieve the same score while having different internal structures.
- **6. More trees do not guarantee higher accuracy.** They often improve stability, but with additional computational cost.
- **7. Test-set size matters.** Achieving 100% on 30 samples is not proof of perfect generalization.

My main takeaway

> The most useful Random Forest is not necessarily the one with the most trees or the highest individual-tree accuracy. It is the one that combines sufficiently strong trees in a way that allows their predictions to complement one another.

The important step in my learning was moving from observing accuracy scores to asking why the algorithm behaves as it does, how randomness changes tree structures, and how the ensemble can correct individual errors. That is the difference between simply using a machine learning algorithm and understanding its mechanics.